

Session 2

AI-Powered Firmware Development



Let the AI write the first draft. You review it, check it and make it smaller.

Part	Topic	Time
01	From a sentence to firmware in INCIPE Workspace	6 min
02	Reviewing what the AI wrote	8 min
03	Making it smaller: memory	8 min
04	When to trust AI, and when to write it yourself	4 min
05	Practice: the climate reader	4 min + homework

Seven words you will hear today.

Word	Plain meaning
Firmware	The program that runs on the board itself, not on your computer.
Compile	Translate the code you can read into instructions the board can run.
Function	A named block of code that does one job. You call it by its name.
Variable	A labelled box that holds one value, like temperature.
Memory (RAM)	The board's short-term workspace, where variables live while the program runs. It is small.
Byte	The unit memory is counted in. One letter of text takes about one byte.
Code review	Reading code line by line to find mistakes before it runs.

Describe, draft, verify, upload.

Describe

The AI chat

Say what the board should do, in one or two sentences.



Draft

The code editor

The AI writes the firmware into your project.



Verify

Top bar

Compiles your code and reports the first mistake it finds.

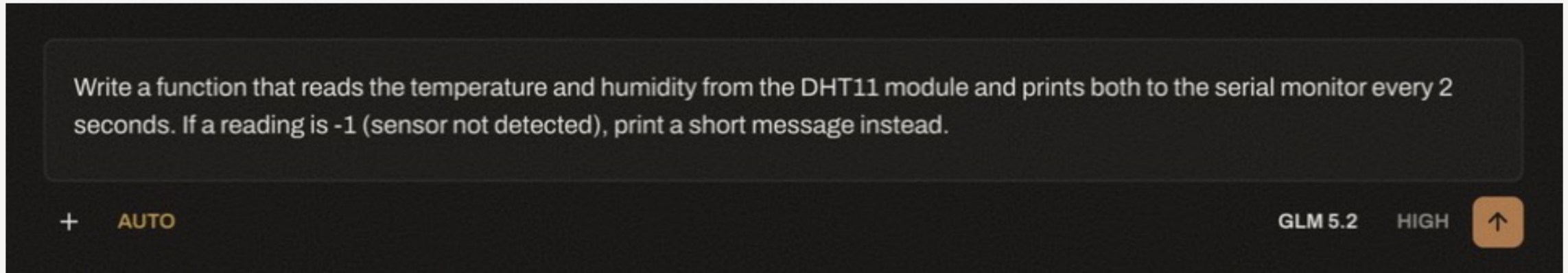


Upload

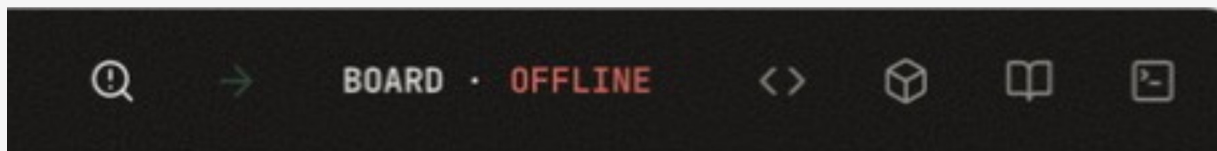
Next to Verify

Compiles and sends the code to the selected board. Read it in the Console.

Describe it in the chat, then Verify and Upload.



The AI chat. Today's prompt: read the DHT11, print both values every 2 seconds, and print a short message if a reading is -1.



The top bar, left to right: Verify, Upload, the Board menu, then the Code editor, Artifacts, Skills and Console panels.

Compiles is not the same as correct.

The Workspace AI builds with Incipe's own toolchain, so its code either compiles or it does not. That catches spelling and grammar mistakes in the code. It cannot tell whether the code does what you meant.

One line of code for each value.

ONE LINE OF CODE

```
float temperature = incipe.getTemperature();  
float humidity = incipe.getHumidity();
```

What you get	<code>incipe.getTemperature()</code>	<code>incipe.getHumidity()</code>
Value	A float in degrees Celsius (°C)	A float, % relative humidity, 0–100
Not detected	-1	-1
Read it every	2000 ms	2000 ms

A float is a number with a decimal point, like 23.4. Full details: [Wiki › Temperature & humidity sensor](#).

A typical first draft. It compiles, and it works — sometimes.

AI DRAFT

```
1 String readClimate() {  
2     double temperature = incipe.getTemperature();  
3     double humidity = incipe.getHumidity();  
4     String report = "Temperature: " + String(temperature) + " C, Humidity: " + String(humidity) + " %";  
5     return report;  
6 }
```

Line	What it does
1	Starts a function called readClimate that hands back a String (a piece of text).
2–3	Asks the sensor for each value and keeps it in a double box.
4	Glues words and both numbers into one long piece of text called report.
5–6	Hands that text back to whoever called the function, and ends it.

Review the draft before you trust it.

#	Ask	This draft
1	Does every incipe.* name match the Wiki exactly?	Yes — <code>getTemperature()</code> , <code>getHumidity()</code>
2	Does it handle <code>-1</code> (sensor not detected)?	No. It would print <code>Temperature: -1.00 C</code> as if it were real.
3	Do the types match what the function returns?	No. The Wiki says <code>float</code> ; the draft uses <code>double</code> .
4	Does it read at the right speed?	Not decided here — we will run it every 2000 ms.
5	Does it keep anything it does not need?	Yes — the whole sentence is stored as text first.

A wrong name is the easiest mistake: Verify stops at it. The `-1` mistake and the memory waste both compile without a complaint — only a human review finds them.

A float takes a 4-byte locker. A double takes 8.

The board's memory is a short row of lockers. A double holds extra decimal places — but the sensor only ever hands us a float, so the extra space holds nothing.

A String is a stretchy bag: every time you glue more words on, it asks for a new, bigger space and copies everything across. Every two seconds, on a small board, that wastes memory.

Item	AI draft	Optimised
Temperature	double — 8 bytes	float — 4 bytes
Humidity	double — 8 bytes	float — 4 bytes
The sentence	Built as one String every call	Not stored — printed piece by piece

Sizes for the INCIPE Board's ESP32: float 4 bytes, double 8 bytes.

The optimised printClimate().

```
1 void printClimate() {  
2     float temperature = incipe.getTemperature();  
3     float humidity = incipe.getHumidity();  
4     if (temperature == -1 || humidity == -1) {  
5         Serial.println("DHT11 not detected yet");  
6         return;  
7     }  
8     Serial.print("Temperature: ");  
9     Serial.print(temperature);  
10    Serial.print(" C, Humidity: ");  
11    Serial.print(humidity);  
12    Serial.println(" %");  
13 }
```

LINE 1 · STEP 1 OF 10

void means the function hands nothing back — it prints instead of returning text.

VARIABLES

None yet.

SERIAL MONITOR

The optimised printClimate().

```
1 void printClimate() {  
2   float temperature = incipe.getTemperature();  
3   float humidity = incipe.getHumidity();  
4   if (temperature == -1 || humidity == -1) {  
5     Serial.println("DHT11 not detected yet");  
6     return;  
7   }  
8   Serial.print("Temperature: ");  
9   Serial.print(temperature);  
10  Serial.print(" C, Humidity: ");  
11  Serial.print(humidity);  
12  Serial.println(" %");  
13 }
```

LINE 2 · STEP 2 OF 10

Ask the sensor for the temperature. Say it is 23.4 °C. A **float** is all we need.

VARIABLES

temperature = 23.4

SERIAL MONITOR

The optimised printClimate().

```
1 void printClimate() {  
2     float temperature = incipe.getTemperature();  
3     float humidity = incipe.getHumidity();  
4     if (temperature == -1 || humidity == -1) {  
5         Serial.println("DHT11 not detected yet");  
6         return;  
7     }  
8     Serial.print("Temperature: ");  
9     Serial.print(temperature);  
10    Serial.print(" C, Humidity: ");  
11    Serial.print(humidity);  
12    Serial.println(" %");  
13 }
```

LINE 3 · STEP 3 OF 10

Ask for the humidity: 51 %. Another 4-byte float.

VARIABLES

temperature = 23.4

humidity = 51.0

SERIAL MONITOR



The optimised printClimate().

```
1 void printClimate() {  
2     float temperature = incipe.getTemperature();  
3     float humidity = incipe.getHumidity();  
4     if (temperature == -1 || humidity == -1) {  
5         Serial.println("DHT11 not detected yet");  
6         return;  
7     }  
8     Serial.print("Temperature: ");  
9     Serial.print(temperature);  
10    Serial.print(" C, Humidity: ");  
11    Serial.print(humidity);  
12    Serial.println(" %");  
13 }
```

LINE 4 · STEP 4 OF 10

|| means "or". Is either value -1? Neither is, so we skip lines 5 and 6.

VARIABLES

temperature = 23.4

humidity = 51.0

SERIAL MONITOR



The optimised printClimate().

```
1 void printClimate() {  
2     float temperature = incipe.getTemperature();  
3     float humidity = incipe.getHumidity();  
4     if (temperature == -1 || humidity == -1) {  
5         Serial.println("DHT11 not detected yet");  
6         return;  
7     }  
8     Serial.print("Temperature: ");  
9     Serial.print(temperature);  
10    Serial.print(" C, Humidity: ");  
11    Serial.print(humidity);  
12    Serial.println(" %");  
13 }
```

LINE 8 · STEP 5 OF 10

Print the label. **print** stays on the same line.

VARIABLES

temperature = 23.4

humidity = 51.0

SERIAL MONITOR

Temperature:

The optimised printClimate().

```
1 void printClimate() {
2   float temperature = incipe.getTemperature();
3   float humidity = incipe.getHumidity();
4   if (temperature == -1 || humidity == -1) {
5     Serial.println("DHT11 not detected yet");
6     return;
7   }
8   Serial.print("Temperature: ");
9   Serial.print(temperature);
10  Serial.print(" C, Humidity: ");
11  Serial.print(humidity);
12  Serial.println(" %");
13 }
```

LINE 9 · STEP 6 OF 10

Print the number. Arduino prints a **float** with two decimal places: 23.40.

VARIABLES

temperature = 23.4

humidity = 51.0

SERIAL MONITOR

Temperature: 23.40

The optimised printClimate().

```
1 void printClimate() {
2   float temperature = incipe.getTemperature();
3   float humidity = incipe.getHumidity();
4   if (temperature == -1 || humidity == -1) {
5     Serial.println("DHT11 not detected yet");
6     return;
7   }
8   Serial.print("Temperature: ");
9   Serial.print(temperature);
10  Serial.print(" C, Humidity: ");
11  Serial.print(humidity);
12  Serial.println(" %");
13 }
```

LINE 10 · STEP 7 OF 10

More label text, still on the same line.

VARIABLES

temperature = 23.4

humidity = 51.0

SERIAL MONITOR

Temperature: 23.40 C, Humidity:

The optimised printClimate().

```
1 void printClimate() {  
2     float temperature = incipe.getTemperature();  
3     float humidity = incipe.getHumidity();  
4     if (temperature == -1 || humidity == -1) {  
5         Serial.println("DHT11 not detected yet");  
6         return;  
7     }  
8     Serial.print("Temperature: ");  
9     Serial.print(temperature);  
10    Serial.print(" C, Humidity: ");  
11    Serial.print(humidity);  
12    Serial.println(" %");  
13 }
```

LINE 11 · STEP 8 OF 10

Print the humidity, also with two decimals: 51.00.

VARIABLES

temperature = 23.4

humidity = 51.0

SERIAL MONITOR

Temperature: 23.40 C, Humidity: 51.00

The optimised printClimate().

```
1 void printClimate() {
2     float temperature = incipe.getTemperature();
3     float humidity = incipe.getHumidity();
4     if (temperature == -1 || humidity == -1) {
5         Serial.println("DHT11 not detected yet");
6         return;
7     }
8     Serial.print("Temperature: ");
9     Serial.print(temperature);
10    Serial.print(" C, Humidity: ");
11    Serial.print(humidity);
12    Serial.println(" %");
13 }
```

LINE 12 · STEP 9 OF 10

println adds the last piece and then starts a new line.

VARIABLES

temperature = 23.4

humidity = 51.0

SERIAL MONITOR

Temperature: 23.40 C, Humidity: 51.00 %

The optimised printClimate().

```
1 void printClimate() {  
2     float temperature = incipe.getTemperature();  
3     float humidity = incipe.getHumidity();  
4     if (temperature == -1 || humidity == -1) {  
5         Serial.println("DHT11 not detected yet");  
6         return;  
7     }  
8     Serial.print("Temperature: ");  
9     Serial.print(temperature);  
10    Serial.print(" C, Humidity: ");  
11    Serial.print(humidity);  
12    Serial.println(" %");  
13 }
```

LINE 13 · STEP 10 OF 10

The function ends. Next call, it reads fresh values.
Run it every 2000 ms from a user thread.

VARIABLES

temperature = 23.4

humidity = 51.0

SERIAL MONITOR

Temperature: 23.40 C, Humidity: 51.00 %

Trust it to multiply. Check that you typed the right numbers.

Trust the AI to...	Check it yourself when...	Write it yourself when...
Write a first draft fast	It uses an incipe.* name — compare it with the Wiki	You cannot explain what a line does yet
Explain an error message	It reads a sensor — is -1 handled?	The fix is one or two lines
Suggest a review checklist	It picks a type or a timing — does it match the Wiki?	You are learning that idea for the first time
Draft comments and a README	It builds text or keeps lists of values	Safety matters: anything that moves or gets hot

Reusable prompt: Review this function line by line. List anything that wastes memory and anything that breaks if the sensor returns -1. Do not rewrite it.

Prompt, verify, review, fix by hand, upload.

1. Open a new project in INCIPE Workspace and send today's prompt.
2. Click Verify. If it fails, ask the AI: "Explain this error in one sentence. Do not rewrite the code."
3. Review the draft with the five questions. Write down every "No".
4. Fix them by hand: double to float, add the `-1` check, print instead of building a String.
5. Verify again, then Upload and watch the Console. Unplug the sensor: you should see DHT11 not detected yet.

Generate it with AI, then optimise the memory by hand.

Step	Do this	Hand in
1	Prompt the Workspace AI for a temperature & humidity read function, using Role, Goal, Context from Lesson 1.	Your prompt
2	Copy the AI's first draft, unchanged.	Draft A
3	Mark every line that uses more memory than it needs, and say why in one sentence.	Your notes
4	Rewrite it by hand so it uses float, stores no String, and handles -1.	Draft B
5	Fill in the table: bytes used by the variables in Draft A vs Draft B.	The table

Temperature is 23.4 and humidity is -1. What prints?

READ THE CODE

```
float temperature = incipe.getTemperature();
float humidity = incipe.getHumidity();
if (temperature == -1 || humidity == -1) {
    Serial.println("DHT11 not detected yet");
    return;
}
Serial.print("Temperature: ");
...
```

ANSWER

DHT11 not detected yet

WHY

23.4 is fine, but humidity is -1, so the "or" test is true. The function prints the message and returns before any numbers.

Temperature is 18 and humidity is 60. What prints?

READ THE CODE

```
Serial.print("Temperature: ");  
Serial.print(temperature);  
Serial.print(" C, Humidity: ");  
Serial.print(humidity);  
Serial.println(" %");
```

ANSWER

**Temperature: 18.00 C,
Humidity: 60.00 %**

WHY

Neither value is -1, so the whole line prints. Arduino prints each float with two decimal places.

Checkpoint 3

Two doubles become two floats. How many bytes are saved?

READ THE CODE

```
// AI draft
double temperature = incipe.getTemperature();
double humidity = incipe.getHumidity();

// Optimised
float temperature = incipe.getTemperature();
float humidity = incipe.getHumidity();
```

ANSWER

8 bytes

WHY

Two doubles = $2 \times 8 = 16$ bytes. Two floats = $2 \times 4 = 8$ bytes.
 $16 - 8 = 8$.

The AI wrote this line. What happens when you click Verify?

READ THE CODE

```
float t = incipe.readTemperature();
```

ANSWER

Verify stops with an error at that line

WHY

There is no `readTemperature()`. The Wiki's name is `incipe.getTemperature()`, and names must match exactly.

Checkpoint 5

A classmate reads the DHT11 every 500 ms "for fresher data". Right?

READ THE CODE

```
printClimate();  
vTaskDelay(pdMS_TO_TICKS(500));
```

ANSWER

No — read it every 2000 ms

WHY

The Wiki says the DHT11 is a slow sensor: reading it more often than every 1–2 seconds does not give fresher data. It only prints the same values more often.

Three things to take away.

1

The Workspace AI turns a sentence into firmware that compiles. Compiling is not the same as correct.

2

Review every draft: exact incipe.* names, the -1 rule, types and timing from the Wiki, and nothing stored that is not needed.

3

Match the type to the data — float, not double — and print instead of building Strings. Same output, less memory.

Next — Session 3: Terminal commands & environment setup. ls, cd, mkdir and friends, and your first look at Git.